Université de Toulouse
Faculté sciences et ingénierie
Master 1 Signal, Image & Apprentissage Automatique (SIA2)

Unité d'Enseignement : Initiation à la recherche et projet

Segmentation en superpixels par l'algorithme SLIC

Auteurs :

P. DONFACK F. AWOUNANG S. KABA M. KABAWA

Encadrant : A. Herbulot

Année universitaire 2025–2026

Signal, Image & Apprentissage Automatique (SIA2)
May 22, 2026
© 2026

Segmentation en superpixels

P. DONFACK[1], F. AWOUNANG[1], S. KABA[1], and M. KABAWA[1]

Université de Toulouse - M1 SIA2

ABSTRACT

1. Introduction

Le traitement d'images est utilisé dans plusieurs domaines tels que la robotique, l'aéronautique, la vision par ordinateur et le domaine médical. Il permet d'analyser une image dans le but d'extraire des informations utiles. Les méthodes classiques de traitement, traitent l'image pixel par pixel. Par exemple pour une image de taille 512×512, elle contient déjà plus de 260 000 pixels. Ce traitement devient alors très coûteux en temps de calcul et en mémoire.

De plus, un pixel isolé ne contient pas assez d'information; il porte qu'une information très locale. Lorsqu'on prend seul un pixel, il ne permet pas d'identifier la structure globale de l'image ni de reconnaître les objets qu'elle contient. Une solution est de regrouper les pixels en petite régions homogènes appelées superpixels. Ces éléments rassemblent des pixels voisins qui partagent des caractéristiques similaires comme la couleur et la position spatiale qui peuvent ensuite être utiliser pour faire de la segmentation de régions ou de la détection d'objets dans les images.

Au lieu de traiter des centaines de milliers de pixels de façon individuel, on travaille sur quelques centaines de régions cohérentes, ce qui réduit considérablement le temps de calcul. Parmi les méthodes existantes, l'algorithme SLIC se distingue par sa simplicité et sa rapidité.

Dans la première partie de ce travail, nous allons essayer de reconstruire l'algorithme SLIC (Simple Linear Iterative Clustering) en nous basant sur l'article original [1]. Puis, dans un second temps, évaluer ses performances sur différents types d'images et d'applications afin de comprendre comment effectuer un choix optimal de ses deux paramètres (k) et (m).

2. Présentation de l'algorithme SLIC et contributions de l'article

Les méthodes qui existaient avant SLIC ne respectaient pas tous les critères de segmentation, comme les contours. Elles étaient très lentes, consommaient beaucoup de mémoire, avaient des paramètres complexes et ne permettaient pas de contrôler le nombre de superpixels. Certaines étaient aussi sensibles au bruit.

Pour améliorer ces difficultés, Achanta et al. (2011) ont adapté la méthode SLIC qui est basée sur le principe du k-means [2]. SLIC regroupe les pixels en tenant compte de la couleur et de la position, ce qui permet d'obtenir des superpixels plus réguliers et cohérents. SLIC est aussi différente des autres méthodes par sa rapidité et sa simplicité. Elle limite également la recherche autour de chaque centre, ce qui explique sa rapidité.

3. Principe et structure de l'algorithme SLIC

3.1. Principe de SLIC

Le principe de SLIC repose sur la représentation de chaque pixel par un vecteur à cinq dimensions :

$$[l, a, b, x, y]$$

où :

- (l, a, b) sont les composantes couleur dans l'espace CIELAB,
- (x, y) sont les coordonnées spatiales du pixel.

L'algorithme regroupe les pixels en superpixels en minimisant une distance combinée entre couleur et position. La distance utilisée est définie par :

$$D = \sqrt{d_c^2 + \left(\frac{d_s}{S}\right)^2 m^2}$$

avec :

- d_c : distance couleur,
- d_s : distance spatiale,
- S : taille moyenne d'un superpixel,
- m : paramètre de compacité.

Lorsque m est petit, les superpixels suivent mieux les contours ; lorsque m est grand, les superpixels sont plus réguliers et compacts.

3.2. Structure de l'algorithme

L'algorithme SLIC suit une structure itérative en quatre étapes principales, comme présenté dans l'article. La figure **??** résume cette structure sous forme de diagramme.

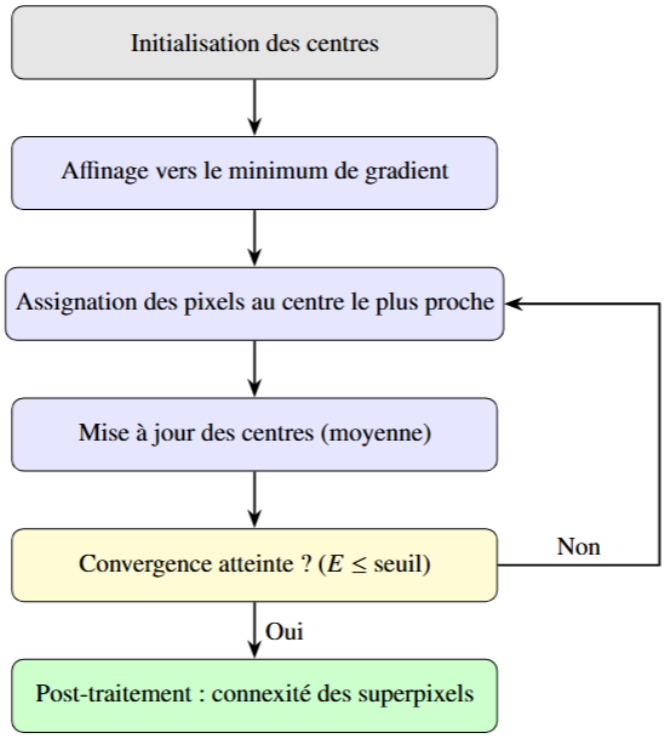


Fig. 1: Diagramme des étapes de l'algorithme SLIC

3.2.1. Initialisation

- Choisir le nombre de superpixels K.
- Calculer l'intervalle :

$$S = \sqrt{\frac{N}{K}}$$

- Initialiser les centres C_k sur une grille régulière.

3.2.2. Affinage des centres

- Pour chaque centre, on examine les 9 pixels de son voisinage 3×3.
- On calcule le gradient de l'image en chaque pixel de ce voisinage.
- Le centre est déplacé vers le pixel qui possède le gradient le plus faible.
- Cette étape évite de placer un centre sur un contour ou un pixel bruité.

3.2.3. Étape d'assignation

Pour chaque centre C_k :

- on considère uniquement une fenêtre locale de taille $2S \times 2S$ autour du centre,
- pour chaque pixel de cette zone, on calcule la distance D,
- le pixel est ensuite affecté au centre le plus proche.

Cette recherche locale rend l'algorithme SLIC très rapide.

3.2.4. Mise à jour des centres

Chaque centre est mis à jour en calculant la moyenne des pixels qui lui sont assignés :

C_k = moyenne des $[l, a, b, x, y]$

3.2.5. Critère d'arrêt

Les étapes d'assignation et de mise à jour sont répétées jusqu'à ce que :

$E \leq$ seuil

En pratique, l'algorithme converge généralement en une dizaine d'itérations.

3.2.6. Post-traitement

Après convergence :

- les pixels isolés sont détectés via un algorithme de composantes connexes,
- chaque pixel isolé est réassigné au label du superpixel voisin le plus proche.

Résumé

En résumé, SLIC fonctionne en plusieurs étapes. D'abord on place les centres sur l'image. Ensuite on assigne chaque pixel au centre le plus proche. Puis on recalcule les centres et on répète jusqu'à ce que l'algorithme converge. À la fin on corrige les pixels qui ne sont pas bien assignés.

Par rapport aux autres méthodes, SLIC est plus rapide car il ne cherche pas dans toute l'image mais seulement autour de chaque centre. Il permet aussi de contrôler le nombre et la forme des superpixels.

4. Implémentation de l'algorithme SLIC

Nous avons implémenté l'algorithme SLIC sous MATLAB, environnement avec lequel nous étions à l'aise. Afin de rendre le code lisible et compréhensible, nous avons choisi de le structurer en plusieurs fonctions distinctes, chacune correspondant à une étape précise de l'algorithme. Nous avons séparé le code en plusieurs fonctions car un seul fichier aurait été trop long. Cela nous a permis de mieux structurer notre travail et de tester chaque étape séparément.

Notre fonction principale est `contratdevie_finale`. Le tableau suivant présente les fonctions implémentées et leur rôle.

Ordre	Fonction	Rôle
1	`SLIC_SIA2`	Fonction principale
2	`initialiser_centres`	Place les centres sur la grille
3	`affiner_centres`	Affine vers le minimum de gradient
4	`calculer_distance`	Calcule la distance D
5	`assigner_pixels`	Assigne chaque pixel au centre le plus proche
6	`recalculer_centres`	Recalcule les centres par moyenne
7	`combler_orphelins`	Corrige les pixels sans label
8	`ges_connexite`	Corrige les pixels isolés
9	`supprimer_petits`	Supprime les petits superpixels

Table 1: Fonctions MATLAB implémentées

Dans les sections suivantes, nous détaillons chaque fonction, son rôle dans l'algorithme et comment nous l'avons implémentée.

4.1. Fonction principale SLIC_SIA2

La fonction `SLIC_SIA2` est la fonction principale qui reçoit en entrée trois paramètres :

- k : est le nombre souhaité de superpixels
- m : est le paramètre de compacité
- image : l'image à segmenter

Elle permet d'appeler toutes les autres fonctions afin de les exécuter dans l'ordre suivant :

1. Conversion de l'image en espace LAB
2. Calcul de $S = \lfloor \sqrt{N/k} \rfloor$
3. Initialisation des centres avec `initialiser_centres`
4. Affinage des centres avec `affiner_centres`
5. Boucle jusqu'à convergence :
   - Assignation des pixels avec `assigner_pixels`
   - Recalcul des centres avec `recalculer_centres`
   - Calcul de l'erreur E
6. Post-traitement :
   - `ges_connexite`
   - `combler_orphelins`
   - `supprimer_petits`

Elle retourne trois éléments :

- `label` : la segmentation finale
- `p` : les positions finales des centres
- `k_reel` : le nombre réel de superpixels obtenus

4.2. Initialisation des centres

Pour implémenter cette étape, nous avons créé la fonction `initialisation_centres_contra_vie`. Elle Reçoit l'image déjà convertie en espace LAB et le nombre souhaité de superpixels k. D'abord la première étape consiste à calculer l'espacement S entre deux centres voisins:

$$S = \left\lfloor \sqrt{\frac{N}{k}} \right\rfloor$$

On utilise deux boucles `for`, l'une pour parcourir les colonnes et l'autre pour parcourir les lignes, afin de placer les centres sur toute l'image. Les boucles démarrent à `floor(S/2)` pour que chaque centre soit placé au milieu de sa zone, puis avancent avec un pas de S dans les deux directions.

Pour chaque position de la grille, on lit les valeurs LAB du pixel et on les stocke avec sa position dans un vecteur de 5 valeurs :

$$[L, A, B, x, y]$$

À la fin, on récupère le nombre réel de centres placés `k_reel` car il peut différer de k selon la taille de l'image.

Nous avons tester cette partie sur une image synthétique de taille 512x512.

La figure 1 image (b) correspond au résultat de la fonction initialisation des centres. Nous avons implémenté avec les centres placés à $k = 500$, sur une distance entre deux centres $S = 22{,}80$. Vu que nous avons calculé S avec `floor` qui fait un arrondissement vers un entier inférieur, $S = 22$. Nous avons choisi `floor` plutôt que `round` car arrondir vers le bas permet d'obtenir un S légèrement plus petit, ce qui permet que les centres couvrent bien toute l'image sans dépasser les bords.

4.3. Affinage des centres

Après avoir fait l'initialisation des centres, nous avons ensuite créé une autre fonction appelée `affiner_centres`. Après l'initialisation, certains centres peuvent tomber sur un contour ou sur un pixel bruité, cette fonction permet donc de déplacer ces centres vers une zone plus homogène.

On commence par calculer le gradient de l'image sur le canal luminance (L) de l'espace LAB avec la fonction `imgradient`. MATLAB utilise par défaut le gradient de Sobel qui va appliquer deux filtres, G_x pour détecter les contours horizontaux, et G_y pour les contours verticaux :

$$G = \sqrt{G_x^2 + G_y^2}$$

Lorsque le gradient est fort cela indique un contour, et lorsqu'il est faible cela indique une zone homogène de l'image.

Ensuite, pour chaque centre on récupère sa position avec `round`, car les coordonnées des centres peuvent être des décimaux après les calculs, et on a besoin d'entiers pour accéder aux pixels de l'image. On parcourt ensuite les 9 pixels de son voisinage 3×3 avec trois boucles `for` imbriquées. On initialise le gradient minimum à l'infini `inf` et on cherche le pixel de gradient le plus faible. On a utilisé la condition `if` pour vérifier que le pixel voisin ne dépasse pas les bords de l'image.

À la fin de cette étape, le centre est finalement déplacé vers le pixel de gradient minimal et ses valeurs LAB sont mises à jour.

La figure 1 (c) montre le résultat après application de l'affinage des centres. Les points jaunes représentent les centres déplacés vers les faibles gradients. Les centres jaunes qui tombent exactement sur les points bleus de l'initialisation ont un faible gradient, donc n'étaient pas situés sur un contour. Par contre, ceux qui ont un léger décalage par rapport au point bleu correspondent aux centres situés sur des contours à fort gradient et sont donc déplacés vers le gradient le plus bas dans leur voisinage.

4.4. Calcul de la distance

Après les étapes précédentes, nous avons créé une fonction appelée `calculer_distance`. Cette fonction calcule trois distances :

- La distance couleur d_c entre le pixel et le centre :

$$d_c = \sqrt{(l - c_L)^2 + (a - c_A)^2 + (b - c_B)^2}$$

- La distance spatiale d_s entre le pixel et le centre :

$$d_s = \sqrt{(x - c_x)^2 + (y - c_y)^2}$$

- La distance finale D qui combine les deux :

$$D = \sqrt{d_c^2 + \left(\frac{m}{S}\right)^2 \times d_s^2}$$

où m est le paramètre de compacité et S est l'espacement entre deux superpixels voisins. Ces deux paramètres seront détaillés dans la fonction principale dans la suite de notre rédaction.

4.5. Assignation des pixels

Vu que dans l'étape précédente les centres étaient placé sur la grille, sans tenir compte du contenu de l'image, cette étape va consister à assigner chaque pixel au centre le plus proche. À la fin de cette étape chaque pixel aura un label qui correspond au centre le plus proche de lui.

Tous les centres sont parcourus afin de récupérer leur position, puis on a défini une fenêtre de $2S \times 2S$ [1] qui est une fenêtre de recherche, tout en définissant un maximum et un minimum afin de ne pas dépasser le bord gauche ou le bord droit de l'image.

Puis tous les pixels de cette fenêtre sont parcourus, fait appel à la fonction `calculer_distance` qui va calculer la distance D entre le pixel et le centre. Si la distance est inférieure à la distance déjà stockée pour ce pixel on fait la mise à jour.

4.6. Recalcul des centres

Après l'étape précédente qui est l'assignation, chaque pixel va appartenir à un superpixel mais le centre ne sera plus forcément au milieu. Donc la fonction `recalculer_centres` va permettre de recalculer le nouveau centre de tous ces superpixels qui ont été formés, et la valeur sera le nouveau centre de ces superpixels.

Cette fonction va donc parcourir tous les pixels, additionner les valeurs, faire une moyenne et ensuite remplacer la valeur du milieu par la nouvelle valeur.

4.7. Combler_orphelins

Pendant l'exécution de l'algorithme, certains pixels peuvent ne pas encore être affectés à une région de superpixels. Cela peut être dû à : des pixels jamais parcourus à cause d'un mauvais choix de fenêtre, du bruit dans l'image ou une mauvaise gestion des bords.

Cette fonction va repérer les pixels non assignés et les rattacher au superpixel voisin dominant (le superpixel majoritaire dans le voisinage).

On retient que un pixel est considéré comme orphelin si, et seulement si, son label vaut zéro ou est négatif [3].

4.8. Gestion de la connexité

Après que l'algorithme s'est arrêté, certains superpixels peuvent être non connectés entre eux et avoir des morceaux qui sont séparés, c'est-à-dire des pixels qui ne sont pas dans leur groupe ou qui sont isolés. On utilise d'abord la 8-connexité, c'est-à-dire les voisins de gauche, droite, en bas, en haut et dans les diagonales, afin de détecter les pixels isolés. On garde le plus grand morceau du superpixel et on réassigne les petits morceaux. Puis on utilise la 4-connexité pour affecter ces pixels au voisin le plus fréquent qui les entoure.

La figure 1 (d) montre l'image après avoir recalculé les centres. Les nouveaux centres sont placés au milieu de leurs pixels assignés. La gestion de la connexité est également appliquée afin de détecter les superpixels non connectés et d'assigner les pixels isolés au voisinage le plus proche.

4.9. Fonction supprimer_petits

Elle interviendra dans le cas d'images bruitées afin d'améliorer la qualité de segmentation.

La fonction va détecter les superpixels trop petits, puis parcourir leurs pixels, ensuite analyser les classes voisines et les affecter à la classe dominante dans son voisinage. Ainsi, les petites régions dont l'aire est négligeable seront fusionnées aux plus grandes régions, ce qui impactera sur le nombre de k superpixels souhaité.

La figure 1 (e) montre une segmentation à $k = 500$ sur une image synthétique d'une fleur, qui est un peu difficile à segmenter. En effet, on a constaté que les centres ne respectaient pas bien les contours de l'image. Pour améliorer le résultat nous avons augmenté le nombre de superpixels à $k = 1200$, comme le montre la figure 1 (f). La segmentation nous a donc donné un meilleur résultat. On retient donc que pour les objets qui ont des détails marqués ou un peu complexes, il faut utiliser un k plus grand.

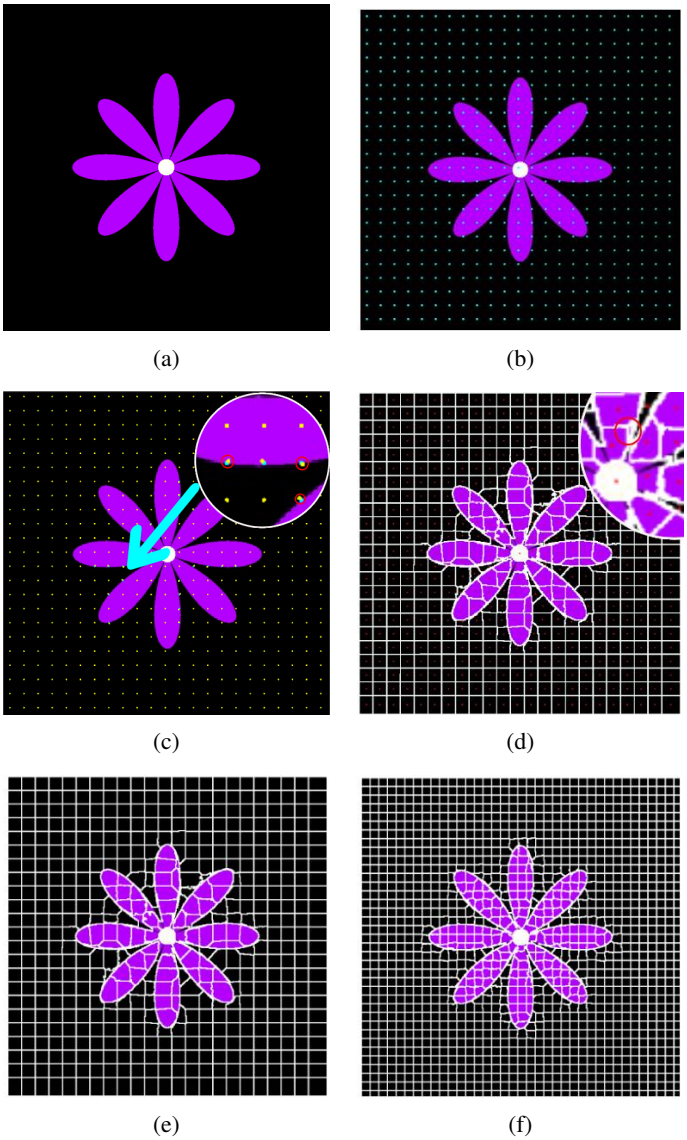


Fig. 2: Visualisation des différentes étapes d'implémentation de notre algorithme SLIC. (a) Image initiale. (b) Initialisation des centres. (c) Affinage des centres vers le gradient le plus bas pour les centres situés sur un contour. (d) Première version de superpixels avant la gestion de la connexité. (e) Résultat avec $K = 500$. (f) Résultat avec $K = 1200$.

5. Résultats

5.1. *Présentation du dataset*

Le dataset Berkeley Segmentation Dataset (BSDS) est composé de 500 images de la vie courante, parmi lesquelles 200 sont réservées à la section de test. Chaque image dispose d'au moins 5 vérités terrain annotées manuellement par des humains.

Nous observons sur l'image du petit garçon figure 4 que selon la personne qui effectue la segmentation, nous obtenons des vérités terrain différentes.

GT1 : l'auteur considère plusieurs régions à savoir le chapeau, les herbes et le panier.

Par contre sur GT3, le chapeau représente une région, le petit garçon et le panier représentent une seule et même région, tandis que le fond représente également une région.

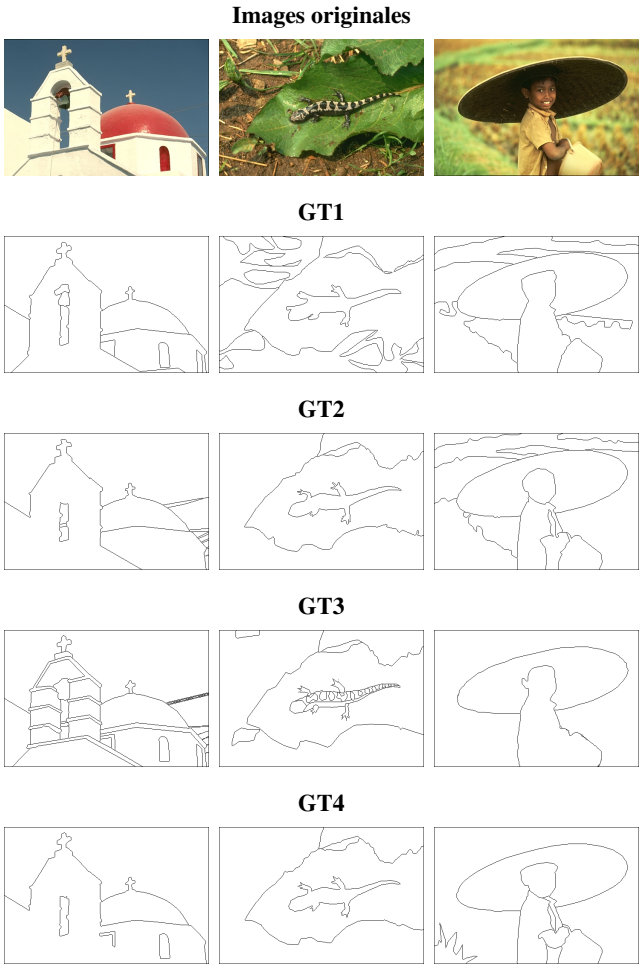


Fig. 3: Cette figure présente trois images dans la base (dataset BSDS500) et affiche 4 exemple de vérité terrain des contours des objets contenus dans ces images, chaque ligne présente donc une vérité faite par un humain.

5.2. *Métrique d'évaluation*

En général, l'évaluation de la qualité d'une segmentation en superpixels se base davantage sur l'adhérence aux contours contenus dans l'image. Les auteurs [1] utilisent la vérité terrain fournie par la base de données BSDS300 [4] pour effectuer une comparaison. Cette notion a été davantage formalisée en 2004 dans [5], déjà présente dans les travaux du BSDS.

5.2.1. **Boundary Recall**

Métrique permettant de savoir si les contours des superpixels capturent bien les vrais contours de l'image.

Pour chaque pixel de contour dans la vérité terrain, on vérifie avec une tolérance de 2. Si un pixel existe sur le contour, alors il est compté comme vrai positif, sinon il est compté comme faux négatif.

Le Boundary Recall est défini comme suit [1] :

$$BR = \frac{\text{vrais positifs}}{\text{vrais positifs + faux négatifs}}$$

5.2.2. Under-Segmentation Error (UE)

L'erreur de sous-segmentation permet d'évaluer si les superpixels débordent les contours des régions [6]:

$$UE(\mathcal{S}) = \frac{\sum_i \sum_k \min(|s_k \cap g_i|, |s_k - g_i|)}{\sum_i |g_i|}$$

Lorsque la valeur est proche de 1 la segmentation n'est pas bonne, et lorsqu'elle est proche de 0 la segmentation est bonne.

Pour notre évaluation nous avons choisi l'ASA car l'erreur de sous-segmentation pénalise les grands superpixels, plus le superpixel est grand, plus il risque de déborder sur plusieurs régions, donc plus l'erreur sera grande.

5.2.3. **Achievable Segmentation Accuracy (ASA)**

L'ASA donne la précision maximale atteignable de la segmentation. Elle mesure la fraction de pixels de la vérité terrain qui sont correctement labelisés par les superpixels [6] :

$$ASA(\mathcal{S}) = \frac{\sum_k \max_i |s_k \cap g_i|}{\sum_i |g_i|}$$

où g_i est une région de la vérité terrain, s_k est un superpixel et $|g_i|$ représente le nombre de pixels dans g_i.

Lorsque l'ASA est proche de 1 on a une bonne segmentation, et lorsqu'elle est proche de 0 on a une mauvaise segmentation.

5.3. *Benchmark*

Le benchmark représente un ensemble de fonctions combinées pour faciliter les expériences et l'évaluation de l'algorithme. À partir du benchmark original fourni avec le dataset Berkeley BSDS500, qui présente une route à suivre décrivant dans quel format sont leurs différentes données d'images et vérités terrain et comment les utiliser, cela associe également des fonctions déjà codées comme l'évaluation du Recall sur une image ou sur un ensemble d'images, le plot de courbes et bien d'autres. Nous l'avons donc modifié pour répondre à nos besoins et pouvoir effectuer l'ensemble des expériences souhaitées.

5.4. *Évaluation qualitative*

Nous avons effectué une comparaison visuelle avec les mêmes images de l'article, figure 3, avec $k = 500$ et $k = 1500$, $m = 10$ et seuil= 0,55 sur le dataset Berkeley.

Visuellement il y a une ressemblance, mais après analyse nous avons constaté que nous n'avons pas exactement les mêmes

résultats. Cela peut s'expliquer par le fait que l'article [**?**] mentionne une étape de post-traitement mais n'explicite pas comment est gérée la connexité.

Pour $k = 500$, l'article arrive mieux à segmenter la croix en une seule région, alors que notre segmentation la divise en deux. On observe cependant sur l'image du chapeau que leur segmentation comporte des superpixels qui encadrent complètement d'autres superpixels, ce qui montre que leur gestion de la connexité tend à réunir tout un superpixel. Notre implémentation consiste à réassigner ces pixels au centre de superpixel le plus proche, évitant ainsi d'avoir des superpixels avec des régions étendues sur d'autres superpixels.

Pour $k = 1500$, les segmentations se ressemblent davantage et les objets sont mieux segmentés. Les contours du chapeau et de l'église sont mieux définis par rapport au k précédent. Sur l'image de l'église, la croix est segmentée en deux dans l'article, alors que dans notre segmentation on observe une zone sombre due à la présence de très petits superpixels.

D'un point de vue qualitatif, l'article produit des superpixels visuellement bien formés. Cependant notre implémentation respecte mieux la connexité grâce aux fonctions `ges_connexite` et `supprimer_petits`, ce qui rend notre segmentation plus robuste.

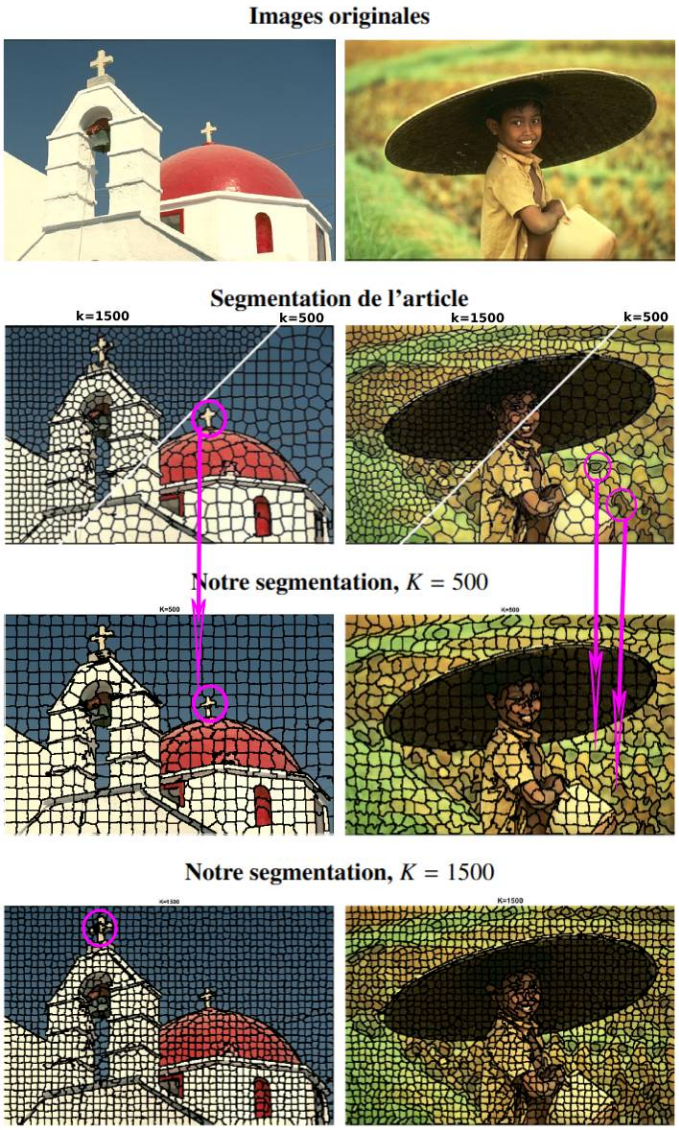


Fig. 4: Comparaison qualitative, la premiere bande presente les deux image originale, la deuxieme presente les secmentation de l'article avec deux valeur de k (500 et 1500), les deux autre bande presente le resultat de notre algorithme sur ces meme images avec les meme valeur de k (500 et 1500) pour $m = 10$.

6. Expériences sur l'influence des paramètres

6.1. ***Influence du paramètre*** k

Nous avons évalué l'influence du paramètre k sur une image de 321×481 provenant de la data set BERKELEY, représentant un lézard, en faisant varier $k\{50, 500, 1000\}$ figure 5.

Comme la vérité terrain a été réalisée par différents annotateurs humains, chaque GT capture des régions différentes. Nous avons donc calculé l'ASA pour chaque vérité terrain puis calculé la moyenne, comme présenté dans le tableau figure 6.

Pour $k = 50$, les superpixels sont trop grands et l'intensité de couleur varie beaucoup au sein d'un même superpixel. Les contours ne sont pas bien suivis, ce qui se confirme par une ASA moyenne de 0,824.

Pour $k = 500$, les superpixels suivent mieux les contours et sont plus compacts et réguliers. L'ASA moyenne s'améliore à 0,933.

Pour $k = 1000$, les contours sont encore mieux définis par les superpixels et plus compacts. L'ASA moyenne atteint 0,960.

Le choix de k dépend de la complexité de l'image : pour une image avec des détails fins, un k grand fonctionne mieux, tandis que pour une image simple un k plus petit suffit.

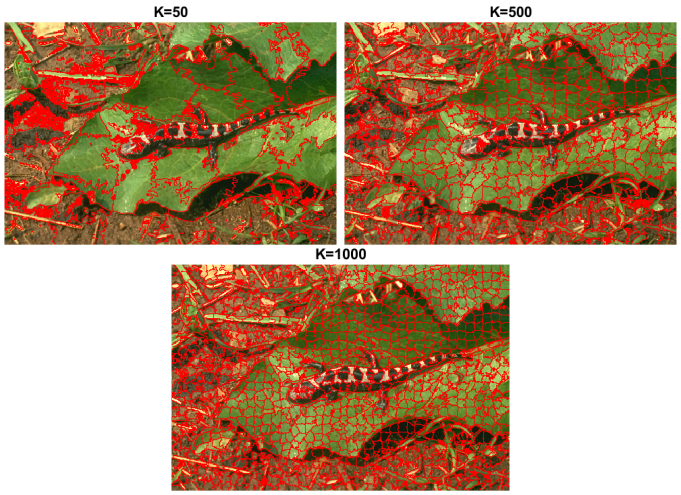


Fig. 5: Application de notre algorithme sur une image avec les valeurs de (k) (50, 500 et 1000).

K	GT1	GT2	GT3	GT4	GT5	Moyenne
50	0.747	0.858	0.825	0.866	0.842	0.824
500	0.866	0.947	0.930	0.964	0.964	0.933
1000	0.923	0.962	0.963	0.980	0.976	0.960

Fig. 6: Évaluation de la segmentation de notre algorithme appliqué à l'image du lézard avec les différentes vérités terrain.

6.2. *Influence du Paramètre m*

D'après la figure 7, nous notons qu'au fur et à mesure que m croit la métrique rappel contour (BR) décroît car Le paramètre m agit sur la similarité des intensités et la proximité spatiale.

Pour m faible, les superpixels suivent davantage les contours, ainsi la segmentation respecte mieux les variations d'intensité. Par contre, quand il est élevé, l'algorithme SLIC donne plus d'importance à la position des pixels dans l'image qu'à leur intensité.

C'est -à -dire que les pixels proches spatialement ont plus de chances d'être regroupés, même si leurs intensités sont un peu différentes. Les superpixels deviennent donc plus semblables en taille.

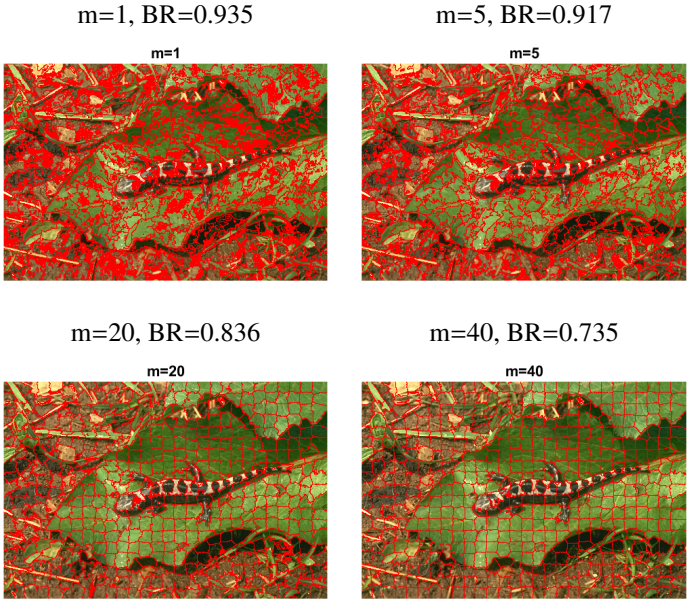


Fig. 7: Application de notre algorithme sur la même image que précédemment avec K=500 fixe et en faisant varier m, afin d'apprécier les performances de l'algorithme et de comprendre l'influence de m.

6.3. *Compromis entre le choix de k et m selon la texture de l'image pour une meilleure segmentation*

Pour des images riches en texture (végétation, zones urbaines...), il faut un k élevé avec un m plutôt faible puisqu'on cherche à capturer les variations locales. Par contre, pour des images peu texturées (homogènes(l'océan, la terre..)) , il faut un k faible, avec un m élevé. Cela produit des superpixels plus réguliers.

6.3.1. Cas avec un seul objet

Nous réalisons ici la même expérience de variation du paramètre m sur une image plus simple qui ne comporte qu'un seul objet cible sur un fond plus ou moins uniforme, afin d'identifier le débordement de contour pour un m grand et le paradoxe entre compacité et adhérence au contour. Les parties identifiées par les cercles en blanc montrent effectivement la dégradation de l'adhérence au contour pour m=20 et m=40, ce qui justifie le fait qu'ils ont un Recall plus faible. Paradoxalement, la segmentation avec m=1 présente une très bonne adhérence au contour mais avec des superpixels très irréguliers. Ceci confirme que le Recall seul ne suffit pas pour évaluer une bonne segmentation. En général, l'algorithme SLIC précède des opérations de traitement qui nécessitent des superpixels[7] assez compacts et réguliers (segmentation sémantique, détection d'objets). Il n'est donc pas judicieux de choisir un m trop petit ni trop grand. Pour ces deux exemples d'images, le compromis se situe donc pour $m = [10, 20]$.

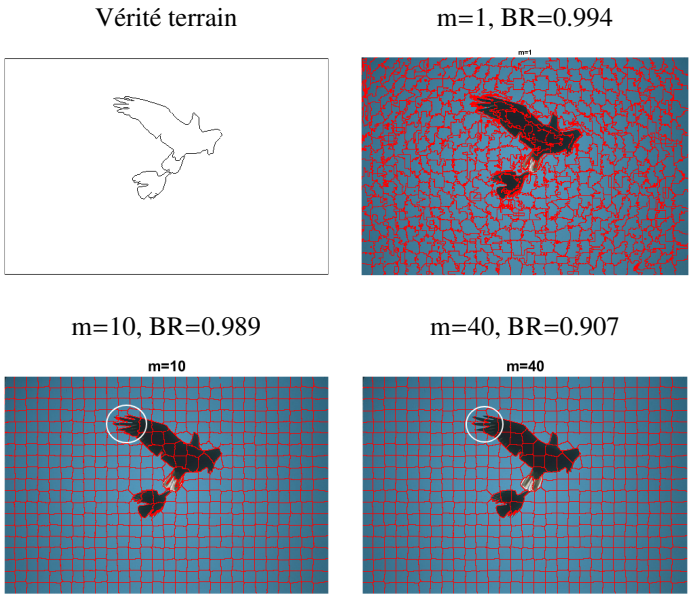


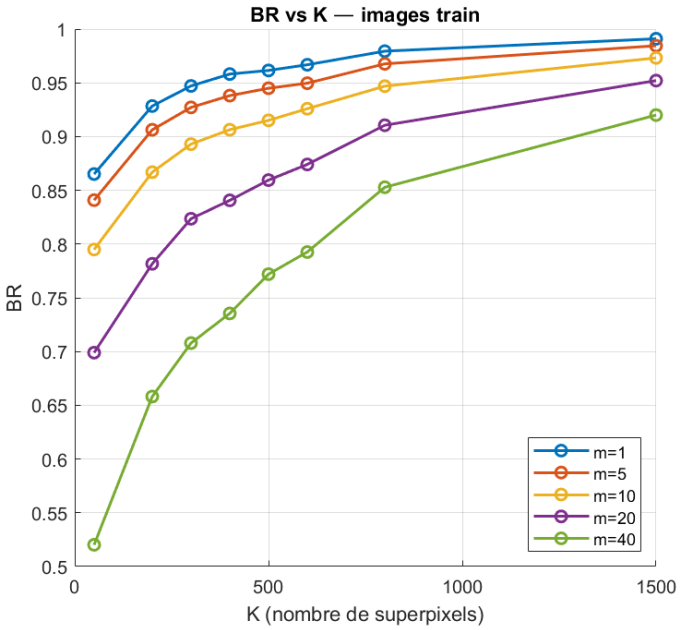
Fig. 8: Même expérience sur la variation du paramètre m avec une image comportant un seul objet cible : un oiseau.

6.3.2. Expérience sur un echantillon d'images de la vie reel

Nous réalisons donc à présent des expériences croisées sur un échantillonnage de 30 images du dataset BSDS500 [4], qui sont des images de la vie réelle choisies aléatoirement, afin de trouver le compromis général entre les paramètres K et m.

Les deux figures ci-dessous confirment les observations faites individuellement sur les expériences précédentes. L'augmentation du nombre de superpixels K permet d'avoir des superpixels qui couvrent davantage les petits détails de l'image, ce qui serait donc approprié pour les images comportant beaucoup de petits détails. Le deuxième graphe de la figure 9 permet quant à lui d'observer l'évolution du BR en fonction de m pour un K fixé. Il montre effectivement qu'un très faible m tend à donner de meilleurs résultats, mais que pour un compromis entre l'adhérence au contour et la compacité des superpixels, il faudrait choisir une valeur intermédiaire, ni trop petite ni trop grande.

la dernière courbe du graphe iso permet donc d'avoir le compromis entre le k et le m , la courbe jaune permet d'avoir le plus grand BR pour un couple de k et m.



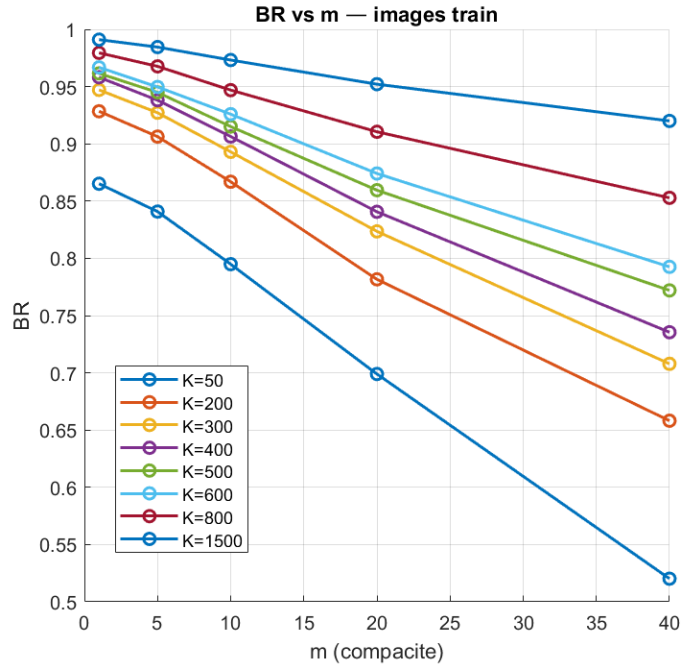


Fig. 9: courbe d'experience presentant l'influence des deux parametre k et m

6.4. *Expérience sur une image IRM*

Nous avons tester notre algorithme sur une image IRM de taille 256 × 256 , provenant de la dataset pixelsRSUA Chest X-Ray Dataset [8]. Les résultats obtenus sont globalement encourageants. Les radiographies pulmonaires traitées présentent un faible contraste, les structures anatomiques s'y distinguent difficilement, leurs intensités étant souvent très proches les unes des

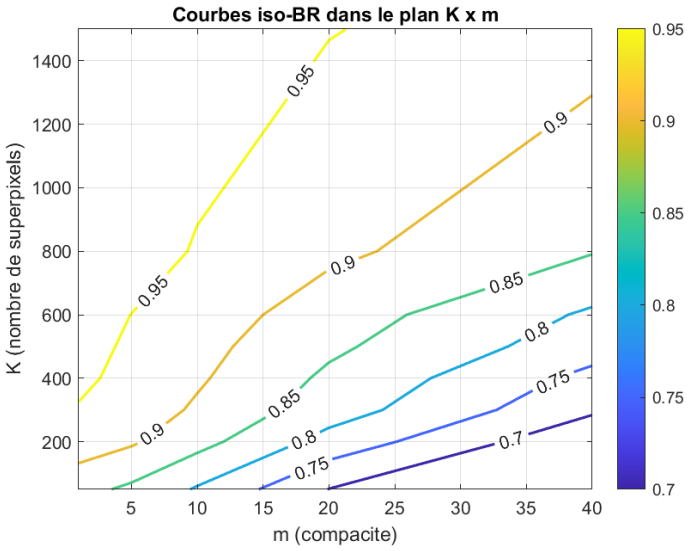


Fig. 10: courbe d'experience presentant l'influence des deux parametre k et m

autres. Dans ce cas, segmenter correctement les contours n'est pas trivial. C'est précisément là que l'algorithme tire son épingle du jeu. Avec un rappel de contour de 0.9588, il parvient à retrouver près de 96% des contours réels définis dans le masque de référence, ce qui traduit une bonne capacité à détecter les frontières pertinentes même lorsque le signal est peu marqué. Dans une application d'aide au diagnostic, où rater une limite anatomique peut avoir des conséquences importantes, ce niveau de performance est particulièrement appréciable.

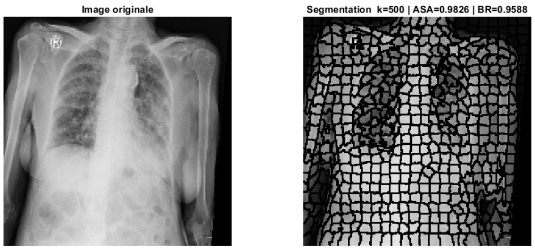


Fig. 11: SLIC sur une image IRM

6.5. *Limites de l'algorithme : Cas particulier d'une image IRM bruitée avec un bruit gaussien centré de variance 0.03.*

Nous avons testé la segmentation avec l'image `brain.png` que nous avons utilisée pendant notre TP3 d'analyse et interprétation des images.

Cette image étant fortement influencée par le bruit, celui-ci modifie aléatoirement les intensités des pixels, ce qui perturbe le regroupement correct des régions.

La segmentation vas s'étaler sur l'image, et n'a plus de sens. Ce qui montre que pour ce type de bruit la segmentation ne fonctionne pas.

On observe une perte de détail, ainsi notre algorithme est donc limité dans ce cas.

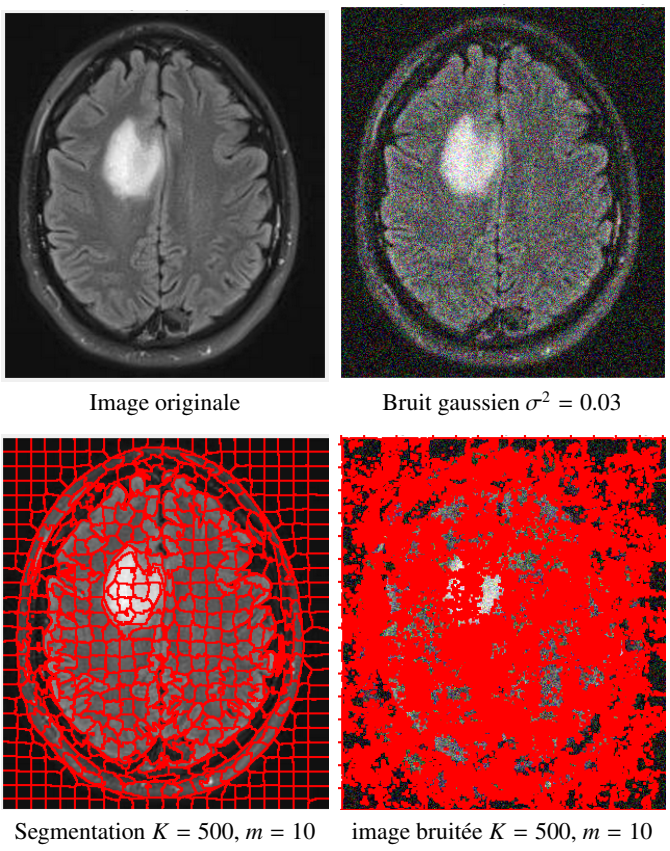


Image originale | Bruit gaussien $\sigma^2 = 0.03$

Segmentation $K = 500$, $m = 10$ | image bruitée $K = 500$, $m = 10$

Fig. 12: Influence du bruit gaussien sur la segmentation SLIC.

6.5.1. Appllication Imagerie spatiale

La segmentation obtenue avec k=1000 et m=10 présente une forte sur-segmentation, avec de nombreux petits superpixels répartis sur toute l'image. Les contours principaux du masque Ground Truth sont globalement bien respectés, traduisant une bonne adhérence aux frontières et un rappel des contours élevé. Cependant, l'excès de superpixels dans les zones homogènes entraîne une fragmentation importante et augmente la complexité de la segmentation. Ces résultats suggèrent que la valeur de k est trop élevée pour cette image ; une réduction de k et de m permettrait d'obtenir des superpixels plus grands et mieux adaptés aux structures réelles de l'image.

7. Conclusion

Après un long travail sur la segmentation en superpixels en utilisant SLIC, nous avons réellement compris comment cette méthode fonctionne.

L'implémentation nous a permis d'assimiler des connaissances en traitement d'images, en programmation MATLAB et en évaluation de méthodes de segmentation.

La qualité de la segmentation dépend principalement du choix de k et de m. Pour résumer : le choix de k dépend de la complexité de l'image et du niveau de détail souhaité, tandis que m contrôle le compromis entre l'adhérence aux contours et la compacité des superpixels. Un m trop petit produit des superpixels irréguliers et fragmentés, tandis qu'un m trop grand produit des superpixels compacts mais qui ne suivent plus les contours. Il faut donc trouver un compromis entre les deux.

Pour les images bruitées, la segmentation ne fonctionne pas car le bruit est etalé sur l'image. Notre fonction

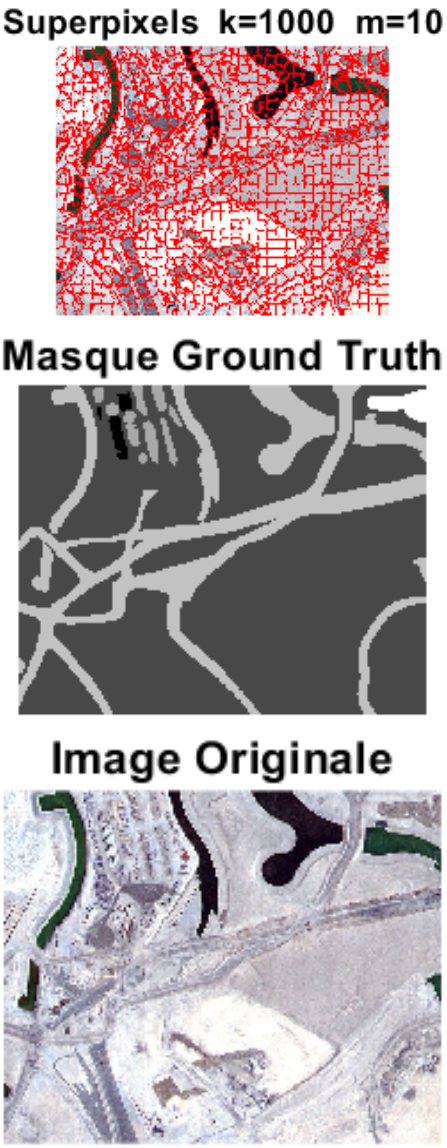


Fig. 13: Segmentation SLIC, masque Ground Truth et image originale

`supprimer_petits` permet d'améliorer le résultat dans ce cas mais dans le cas du bruit gaussien qu'on a tester, ne fonctionne pas.

Pour les images en imagerie spatiale, la très haute résolution implique un très grand nombre de superpixels et de calculs, ce qui donne des résultats moins satisfaisants avec nos paramètres actuels.

Pour les images médicales, le choix de k dépend de ce que l'on souhaite segmenter. Pour segmenter une tumeur ou les côtes sur un poumon, un k plus petit donne un résultat plus convenable.

Concernant la reproductibilité, SLIC est un algorithme déterministe, les mêmes paramètres donnent toujours les mêmes résultats. Cependant nous n'avons pas obtenu exactement les mêmes résultats que l'article de référence, ce qui s'explique par le fait que certains détails d'implémentation notamment le post-traitement n'ont pas été explicités dans l'article.

Pour terminer nous proposons pour quelqu'un qui souhaite utiliser cette méthode un m entre [10 et 20]. Et les valeur dépendrons de la taille de l'image.

References

[1] R. Achanta, A. Shaji, K. Smith, A. Lucchi, P. Fua, and S. Süsstrunk, "SLIC Superpixels Compared to State-of-the-Art Superpixel Methods," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 34, no. 11, pp. 2274–2282, Nov. 2012. [Online]. Available: https://ieeexplore.ieee.org/document/6205760

[2] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, Volume 1: Statistics*. University of California Press, Jan. 1967, vol. 5.1, pp. 281–298. [Online]. Available: https://projecteuclid.org/ebooks/berkeley-symposium-on-mathematical-statistics-and-probability/Proceedings-of-the-Fifth-Berkeley-Symposium-on-Mathematical-Statistics-and/chapter/Some-methods-for-classification-and-analysis-of-multivariate-observations/bsmsp/1200512992

[3] D. Stutz, A. Hermans, and B. Leibe, "Superpixels: An evaluation of the state-of-the-art," *Computer Vision and Image Understanding*, vol. 166, pp. 1–27, Jan. 2018. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1077314217300589

[4] D. Martin, C. Fowlkes, D. Tal, and J. Malik, "A database of human segmented natural images and its application to evaluating segmentation algorithms and measuring ecological statistics," in *Proceedings Eighth IEEE International Conference on Computer Vision. ICCV 2001*, vol. 2, Jul. 2001, pp. 416–423 vol.2. [Online]. Available: https://ieeexplore.ieee.org/document/937655/footnotes

[5] D. Martin, C. Fowlkes, and J. Malik, "Learning to detect natural image boundaries using local brightness, color, and texture cues," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 26, no. 5, pp. 530–549, May 2004. [Online]. Available: https://ieeexplore.ieee.org/document/1273918

[6] X. Wei, Q. Yang, Y. Gong, N. Ahuja, and M.-H. Yang, "Superpixel Hierarchy," *IEEE Transactions on Image Processing*, vol. 27, no. 10, pp. 4838–4849, Oct. 2018. [Online]. Available: https://ieeexplore.ieee.org/document/8360136

[7] X. Xie, G. Xie, X. Xu, L. Cui, and J. Ren, "Automatic Image Segmentation With Superpixels and Image-Level Labels," *IEEE Access*, vol. 7, pp. 10 999–11 009, 2019. [Online]. Available: https://ieeexplore.ieee.org/document/8607985

[8] R. RSUA, "Rsua chest x-ray dataset," 2023. [Online]. Available: https://doi.org/10.17632/2jg8vfdmpm.1

DIAGRAMME DE GANTT - PROJET SEGMENTATION SLIC							
Tâches	Début	Fin	Responsable(s)	P. Donfack	F. Awounang	S. Kaba	M. Kabawa
Première réunion (consignes du projet)	19/02/2026	19/02/2026	Toute l'équipe				
Lecture et comprehension de l'article	22/02/2026	08/03/2026	Toute l'équipe				
Initialisation centres et calcul S	10/03/2026	28/03/2026	P. Donfack	X			
Affinage centres	10/03/2026	29/03/2026	F. Awounang		X		
Assignation pixels	10/03/2026	30/03/2026	S. Kaba			X	
Connexité et suppression petits superpixels	10/03/2026	30/03/2026	M. Kabawa				X
Fusion des travaux	11/04/2026	13/04/2026	Toute l'équipe				
Présentation résultats	14/04/2026	14/04/2026	Toute l'équipe				
Amélioration et tests de l'algorithme	15/04/2026	25/04/2026	Toute l'équipe				
Début rédaction rapport	28/04/2026	17/05/2026	Toute l'équipe				
Finalisation rapport	18/05/2026	22/05/2026	Toute l'équipe				

Fig. 14: Diagramme de Gantt